

Embedded Linux & System Software Interview Handbook

Chapter 7 - Linux Kernel Synchronization & Concurrency (Part 7)

Advanced Topics: Real-World Design Patterns, Interview Case Studies & Best Practices

Synchronization Design Patterns

Choosing the correct synchronization primitive is a critical design decision. Use spinlocks for short non-sleeping sections, mutexes for process context, wait queues for blocking until events, RCU for read-heavy workloads and atomic operations for simple counters and flags.

Decision Matrix

Scenario	Recommended Primitive
Interrupt handler	Spinlock
User-space file operation	Mutex
Hardware event notification	Wait Queue
Reference counter	Atomic variable
Read-mostly shared data	RCU
Per-core statistics	Per-CPU data

Case Study 1: UART Driver

The interrupt handler acquires a spinlock, copies received bytes into a circular buffer, releases the lock and wakes blocked readers using a wait queue.

Case Study 2: Network Driver

NAPI polling reduces interrupt overhead. Per-CPU queues and RCU improve scalability while spinlocks protect short critical sections.

Case Study 3: Block Driver

Request queues are synchronized using fine-grained locking while completions notify waiting threads when I/O finishes.

Performance Best Practices

- Keep critical sections short.
- Reduce lock contention.
- Prefer fine-grained over global locks.
- Avoid unnecessary wake-ups.
- Measure with perf and ftrace before optimizing.

Top Interview Questions

Topic	Typical Question
-------	------------------

Spinlocks	Why can't they sleep?
Mutex	When is it preferred?
RCU	Explain grace period.
Memory Barriers	Why are they required on SMP?
Deadlocks	How do you detect and prevent them?
Wait Queues	Difference from polling?
Atomics	When are they enough without locks?

Final Chapter Summary

This chapter presented a complete guide to Linux kernel synchronization, from basic locking to advanced scalability techniques. Understanding these concepts is essential for writing robust Linux device drivers, BSP software and high-performance kernel components, and they are frequently evaluated in senior Embedded Linux interviews.